

DKG-TGAT-Forecast: Context-Aware Temporal Knowledge Graph Attention for Multi-Horizon Failure-Rate Prediction in Kubernetes and Hadoop Platforms

A new method synthesized from Paper 3 and Paper 4

Full SCI-Style Research Manuscript Draft

Recommended Pair 1: Paper 3 + Paper 4

Author name(s) omitted for review

Affiliation and corresponding-author information to be added

Research integrity note: This manuscript proposes a new unified method. Numerical results attributed to the source studies are clearly identified as prior evidence; no experimental results for the proposed unified model are fabricated.

Abstract

Modern big-data platforms built on Kubernetes, Hadoop, and microservice ecosystems generate large volumes of telemetry while continuously changing their execution topology. Pods scale, workloads migrate, services are redeployed, HDFS blocks are re-replicated, and network dependencies evolve. Conventional anomaly detectors generally process flat time-series vectors or assume a static dependency graph, which limits their ability to distinguish benign topology changes from degradation that precedes a real failure. This paper proposes DKG-TGAT-Forecast, a context-aware deep learning framework that combines a dynamic temporal knowledge graph with graph attention and recurrent sequence modeling to estimate component-level and platform-level future failure rates. The method is motivated by two complementary recent solutions: dynamic knowledge-graph-based AIOps for topology-aware anomaly detection and GAL-MAD, an explainable GAT-BiLSTM autoencoder for microservice anomaly detection. DKG-TGAT-Forecast generalizes these ideas into a single cross-platform model. Kubernetes and Hadoop entities are represented in typed graph snapshots, while Prometheus-style metrics, network observations, scheduler events, storage events, and dependency changes are attached as time-indexed attributes. A relation-aware graph attention encoder learns spatial and semantic dependencies, a Bi-LSTM or temporal Transformer learns degradation trajectories, and multi-task heads jointly estimate current anomaly probability, discrete failure hazard, calibrated failure probability at multiple horizons, and a derived failure-rate measure. An explanation layer combines learned attention, SHAP-style feature attribution, and explicit graph-path reasoning to identify influential components and likely propagation routes. A rigorous validation protocol is specified using controlled fault injection, temporally separated train-validation-test partitions, cross-platform transfer tests, calibration metrics, early-warning lead time, and root-cause localization accuracy. The proposed framework is intended to move AIOps from reactive anomaly detection toward proactive, interpretable, and topology-aware failure prevention in large-scale data platforms.

Keywords: dynamic knowledge graph; graph attention network; Bi-LSTM; failure-rate prediction; Kubernetes; Hadoop; AIOps; anomaly detection; temporal graph; root-cause analysis

1. Introduction

Large-scale digital services increasingly depend on cloud-native and distributed data platforms in which computation, storage, orchestration, and communication are spread across many interacting components. Kubernetes manages containerized workloads through Pods, Deployments, ReplicaSets, Services, Nodes, volumes, and controllers. Hadoop ecosystems coordinate NameNodes, DataNodes, ResourceManagers, NodeManagers, applications, jobs, queues, and HDFS blocks. In both environments, reliability is not determined by the condition of a single machine. It emerges from a changing web of dependencies in which one overloaded component can trigger rescheduling, replication, retries, queue buildup, latency growth, and cascading service degradation.

The operational challenge is therefore temporal and relational. A CPU spike by itself may be harmless, while the same spike combined with rising restart frequency, a shrinking replica set, a saturated parent Node, and a growing service dependency fan-out can indicate a high near-term failure risk. Similarly, a DataNode with increasing I/O latency may remain available for a period, but when combined with missing heartbeats, block-replication pressure, and downstream job retries it can become a leading indicator of platform instability. Flat machine-learning models lose much of this context because they treat each observation as an independent vector.

Recent research provides two complementary pieces of a stronger solution. The dynamic-knowledge-graph approach of Moens et al. demonstrates that evolving microservice context can be represented explicitly and that contextual graph features improve anomaly detection in the presence of autoscaling, rescheduling, and other topology changes. GAL-MAD demonstrates that graph attention and Bi-LSTM sequence modeling can capture spatial service dependencies and temporal behavior while remaining explainable through SHAP-based localization. The central research opportunity is to combine these strengths and move beyond current anomaly detection to explicit multi-horizon failure forecasting.

This paper introduces DKG-TGAT-Forecast. Instead of assuming a fixed adjacency matrix or using only a current graph snapshot, the framework constructs a sequence of dynamic heterogeneous knowledge graphs. Each graph contains typed platform entities, typed operational relationships, component attributes, performance metrics, and contextual events. A relation-aware graph attention network processes each snapshot, and a temporal encoder models the recent sequence of embeddings. The output is not merely a binary anomaly label. The model estimates the probability that a component will fail within defined horizons, the discrete hazard of failure in each future interval, and a comparable failure-rate statistic.

The design is deliberately cross-platform. Kubernetes and Hadoop have different ontologies, but both can be expressed through a common upper-level schema of compute resources, workload resources, storage resources, control resources, communication dependencies, ownership relations, and health observations. This makes it possible to train domain-specific models while sharing architectural principles and selected representation parameters.

The paper makes five main contributions. First, it defines a unified dynamic temporal knowledge graph for Kubernetes and Hadoop reliability analysis. Second, it proposes a heterogeneous relation-aware GAT combined with Bi-LSTM or temporal Transformer modeling. Third, it formulates multi-horizon failure probability and discrete survival hazard as first-class prediction targets. Fourth, it integrates attention, SHAP, and explicit graph traversal for explainable root-cause and propagation reasoning. Fifth, it specifies an experimentally rigorous evaluation design that avoids temporal leakage and measures not only classification quality but also probability calibration, warning lead time, root-cause accuracy, and operational overhead.

2. Background and Related Work

2.1 Dynamic knowledge graphs for AIOps

Dynamic microservice architectures invalidate the simplifying assumption that application topology remains unchanged throughout a monitoring dataset. Autoscaling can create or remove Pods, scheduler decisions can move workloads between Nodes, rolling updates can temporarily change dependency patterns, and infrastructure faults can force rescheduling. The context-aware AIOps method represented by Paper 3 addresses this problem by constructing dynamic knowledge-graph snapshots from resource and network monitoring. The graph explicitly captures services, workloads, nodes, and their changing dependencies. Its evaluation shows that contextual graph features improve anomaly detection relative to non-contextual baselines in a benchmark containing both static and topology-driven dynamic faults.

A particularly valuable aspect of the knowledge-graph approach is interpretability. Explicit neighborhood features can be traced back to relations such as a workload running on a particular node or a service depending on another service. This creates a direct bridge between anomaly scores and operational reasoning. However, the source method primarily evaluates lightweight embedding and unsupervised detection pipelines rather than a deep temporal graph model for future failure prediction.

2.2 Graph attention and temporal anomaly modeling

Paper 4, GAL-MAD, approaches microservice anomalies from a complementary direction. It uses two Graph Attention Network layers to learn service interactions, followed by a bidirectional LSTM to model temporal dependencies. The model is trained as an autoencoder on normal data, and reconstruction error serves as an anomaly signal. In reported experiments on the RS-Anomic dataset, GAL-MAD achieved high recall across multiple anomaly prevalence settings, including recall of 0.9884 for the 95:5 normal-to-anomaly scenario. The study also uses SHAP to identify the most influential anomalous service and feature.

GAL-MAD demonstrates the importance of combining graph structure with time-series modeling. Its static application topology, however, is not sufficient for highly dynamic Kubernetes or Hadoop environments in which the set of vertices and edges can change over time. DKG-TGAT-Forecast therefore replaces the fixed adjacency assumption with dynamic typed graph snapshots and changes the learning target from reconstruction-based anomaly detection to calibrated multi-horizon failure forecasting.

2.3 Failure prediction versus anomaly detection

Anomaly detection and failure prediction are related but not equivalent. An anomaly is a deviation from learned normal behavior. A failure is an operational outcome such as component unavailability, health-check failure, container termination, persistent service-level objective violation, NodeNotReady state, loss of a Hadoop daemon, or inability to complete an expected workload. Many anomalies are transient and never become failures, while gradual degradation can lead to failure without producing an extreme instantaneous outlier.

The proposed method therefore separates current anomaly probability from future failure probability. This distinction supports operational decisions. A high anomaly score with low failure probability may warrant observation, whereas a moderate anomaly score combined with a sharply increasing one-hour failure probability may justify proactive migration or scaling.

3. Research Gap and Proposed Contributions

The literature reveals a gap at the intersection of dynamic context, deep graph learning, time-aware degradation modeling, and explicit reliability forecasting. Dynamic knowledge graphs provide semantically rich and interpretable context, but lightweight anomaly detectors do not fully exploit end-to-end graph representation learning. GAT-BiLSTM architectures capture spatial and temporal dependencies, but a fixed graph does not faithfully represent autoscaling, migration, rescheduling, or dynamic infrastructure relationships. Existing methods also commonly optimize anomaly scores rather than calibrated failure probabilities.

DKG-TGAT-Forecast addresses these gaps through four design principles. The graph is dynamic rather than static; relations are typed rather than homogeneous; temporal history is modeled explicitly rather than collapsed into one snapshot; and the output is probabilistic failure risk rather than only an anomaly class. This formulation supports both Kubernetes and Hadoop while preserving platform-specific ontology detail.

Table 1. Complementarity of the two source solutions and the proposed synthesis

Capability	Paper 3	Paper 4	DKG-TGAT-Forecast
Dynamic topology	Yes	No - static adjacency	Yes - dynamic typed snapshots
Knowledge graph semantics	Yes	No	Yes
Deep graph learning	Limited / lightweight embeddings	Yes - GAT	Yes - relation-aware heterogeneous GAT
Temporal deep model	Limited	Yes - Bi-LSTM	Yes - Bi-LSTM or temporal Transformer
Future failure-rate prediction	No	No	Yes - multi-horizon probability and hazard
Explainability	Explicit KG features	SHAP localization	Attention + SHAP + graph-path reasoning

4. Platform and Graph Formulation

4.1 Dynamic heterogeneous graph sequence

At monitoring time t , the platform is represented as a heterogeneous knowledge graph $G_t = (V_t, E_t, X_t, R)$, where V_t is the set of currently known resources, E_t is the set of typed relationships, X_t contains time-indexed node and edge attributes, and R is the relation vocabulary. The recent operating history is a graph sequence $G_{[t-L+1:t]}$ containing L snapshots.

For Kubernetes, node types include Cluster, Node, Namespace, Deployment, ReplicaSet, StatefulSet, Pod, Container, Service, Ingress, PersistentVolume, and PersistentVolumeClaim. For Hadoop, node types include NameNode, DataNode, ResourceManager, NodeManager, ApplicationMaster, Job, Queue, HDFSBlock, StorageVolume, and Host. A shared upper-level ontology maps these platform-specific entities into ComputeResource, WorkloadResource, StorageResource, ControlResource, NetworkEndpoint, and LogicalService categories.

$$G_t = (V_t, E_t, X_t, R) \quad (1)$$

$$G_{[t-L+1:t]} = \{G_{(t-L+1)}, \dots, G_t\} \quad (2)$$

4.2 Relation vocabulary

Relations are semantically typed because different operational dependencies imply different failure-propagation mechanisms. Kubernetes relations may include RUNS_ON, MANAGED_BY, OWNS, EXPOSES, COMMUNICATES_WITH, MOUNTS, SELECTS, and DEPENDS_ON. Hadoop relations may include STORES, REPLICATES, MANAGED_BY, SCHEDULED_BY, RUNS_ON, SUBMITS_TO, HEARTBEATS_TO, and DEPENDS_ON. Dynamic edges receive validity intervals or snapshot timestamps so that the model can distinguish persistent architecture from transient runtime relationships.

4.3 Component features

Each component receives a typed feature vector composed of resource metrics, event indicators, network measurements, configuration descriptors, and graph-context statistics. Kubernetes features can include CPU saturation, memory working set, throttling, disk pressure, restart count, readiness failures, liveness failures, OOMKilled events, request latency, retransmissions, and replica availability. Hadoop features can include HDFS capacity, missing blocks, corrupt blocks, heartbeat delay, JVM garbage-collection time, RPC latency, queue length, task retries, container failures, and disk I/O latency.

Feature availability differs by node type. Type-specific encoders project raw feature vectors into a common latent space, avoiding the need to force unrelated metrics into one schema.

DKG-TGAT-Forecast Architecture

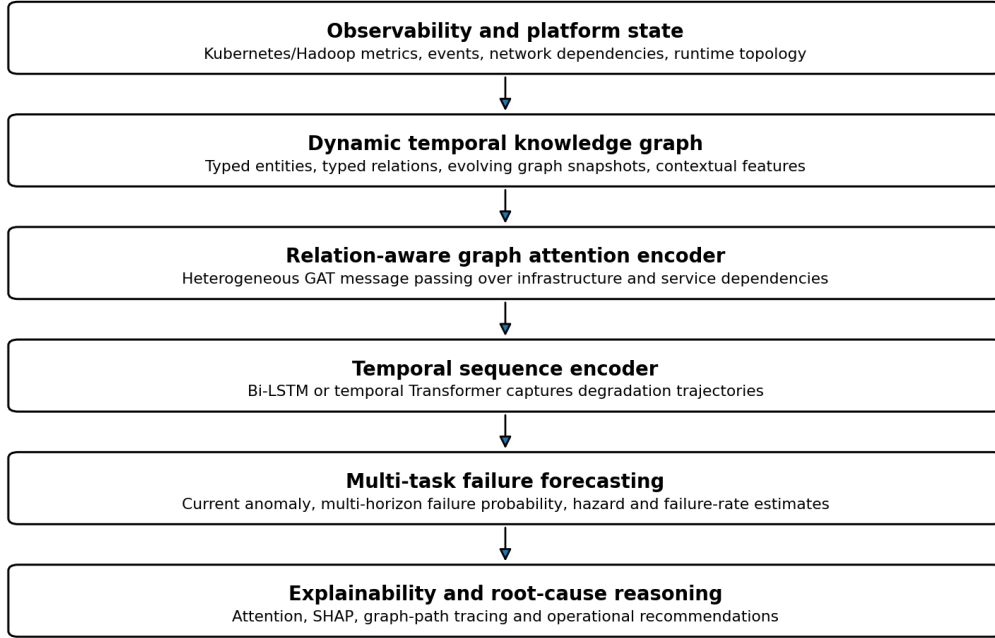


Figure 1. Proposed DKG-TGAT-Forecast processing pipeline.

5. Dynamic Knowledge Graph Construction

The graph-construction pipeline operates continuously. Platform APIs provide resource identity and ownership relations. Prometheus-compatible exporters provide metric time series. Service meshes, eBPF observability tools, or network monitors provide communication edges. Kubernetes Events and Hadoop daemon logs provide explicit operational state transitions. Each observation is mapped to a resource identifier and attached to the corresponding graph entity.

To preserve dynamic context without excessive storage, the system maintains event-driven topology changes and fixed-interval feature snapshots. A topology edge is updated only when its state changes, whereas metric attributes are aggregated over windows such as 15 seconds, one minute, and five minutes. This hybrid representation reduces redundant graph writes while preserving enough temporal resolution for degradation analysis.

The graph can be stored in Neo4j, a temporal RDF store, or an event-sourced graph service. The learning pipeline does not depend on a specific database. What matters is the ability to recover typed neighborhoods and time-aligned graph snapshots.

Table 2. Cross-platform ontology used by the proposed knowledge graph

Entity family	Kubernetes examples	Hadoop examples	Representative features
Compute	Node, virtual machine	Host, NodeManager host	CPU, memory, disk, pressure, temperature
Workload	Pod, Container, Deployment	ApplicationMaster, job, task container	restarts, retries, latency, failures
Storage	PV, PVC, CSI volume	DataNode volume, HDFS block	capacity, I/O latency, replication health
Control	Scheduler, controller	NameNode, ResourceManager	queue delay, RPC latency, leader health
Dependency	Service communication	job/data/daemon dependency	traffic, error propagation, edge criticality

6. Relation-Aware Temporal Graph Attention Model

6.1 Type-specific encoding

For each node v , a node-type-specific encoder ϕ_{τ} transforms raw observations into an initial latent state. This permits heterogeneous feature dimensions while preserving node-type meaning. Missing values are represented by masking indicators so that absence of telemetry is not confused with a measured zero.

$$h_v^{(0,t)} = \phi_{\tau}(v)(x_v^{(t)}, m_v^{(t)}) \quad (3)$$

6.2 Relation-aware graph attention

The graph encoder extends standard GAT by learning relation-specific transformations and attention parameters. Messages arriving through RUNS_ON can therefore be weighted differently from COMMUNICATES_WITH or STORES. For each relation r , the attention score between source u and target v depends on their transformed states, relation embedding, and optional edge attributes such as traffic volume or replica count.

$$e_{uv}^{(r,t)} = \text{LeakyReLU}(a_r^T [W_r h_u^{(t)} \parallel W_r h_v^{(t)} \parallel e_{uv}^{(r,t)}]) \quad (4)$$

$$\alpha_{uv}^{(r,t)} = \exp(e_{uv}^{(r,t)}) / \sum_{k \in N_r(v)} \exp(e_{kv}^{(r,t)}) \quad (5)$$

$$h_v^{(t)} = \sigma(W_0 h_v^{(0,t)} + \sum_r \sum_{u \in N_r(v)} \alpha_{uv}^{(r,t)} W_r h_u^{(0,t)}) \quad (6)$$

6.3 Temporal degradation encoder

Snapshot embeddings are stacked across the historical window. A Bi-LSTM can be used when both directions within the observation window are allowed during offline training, whereas a causal LSTM, GRU, or temporal Transformer is required for strict online inference. The temporal state $z_v^{(t)}$ summarizes recent degradation, recovery, oscillation, and topology changes.

$$z_v^{(t)} = \text{TemporalEncoder}(h_v^{(t-L+1)}, \dots, h_v^{(t)}) \quad (7)$$

6.4 Multi-task prediction heads

The model jointly predicts current anomaly probability, component failure probability at multiple future horizons, and discrete survival hazard. Joint optimization encourages the representation to distinguish current abnormality from trajectories that are likely to become operational failures.

$$\hat{a}_v^{(t)} = \text{sigmoid}(W_a z_v^{(t)} + b_a) \quad (8)$$

$$\hat{p}_{(v, \Delta_j)}^{(t)} = \text{sigmoid}(W_j z_v^{(t)} + b_j) \quad (9)$$

$$\hat{q}_{(v,k)}^{(t)} = P(T_v = k \mid T_v \geq k, G_{[t-L+1:t]}) \quad (10)$$

7. Failure Probability, Hazard, and Failure Rate

For a future horizon Δ , the primary prediction is the probability that component v will experience a defined failure before $t + \Delta$, conditioned on being operational at time t . A failure label is derived from objective operational outcomes rather than from anomaly score alone. Examples include Pod eviction, repeated failed health probes followed by unavailability, NodeNotReady, unrecoverable container termination, sustained SLO violation, loss of a Hadoop daemon, repeated task failure beyond policy, or HDFS storage unavailability.

The discrete survival formulation handles right-censored observations. Many resources remain healthy throughout the dataset, so forcing them into a simple positive/negative classification at every horizon wastes time-to-event information. The cumulative probability of failure by interval K is obtained from the product of conditional survival probabilities.

$$P(T_v \leq K) = 1 - \prod_{k=1..K} (1 - \hat{q}_{(v,k)}^{(t)}) \quad (11)$$

$$\lambda_{(v, \Delta)} = -\ln(1 - \hat{p}_{(v, \Delta)}^{(t)}) / \Delta \quad (12)$$

The derived λ value is an interval-specific equivalent failure rate under an exponential approximation. It is useful for ranking and aggregation, but the model does not assume that real failures follow a globally exponential distribution. Platform-level risk can be calculated as a criticality-weighted aggregation so that control-plane resources and highly connected storage nodes have greater operational influence than disposable low-priority workloads.

$$\Lambda_{\text{platform}}(\Delta) = \sum_v w_v \lambda_{(v, \Delta)} / \sum_v w_v \quad (13)$$

8. Explainability, Root-Cause Localization, and Propagation

Explainability is essential for operational use. A platform engineer must know why a failure probability increased and which component should be inspected first. DKG-TGAT-Forecast combines three explanation channels.

First, relation-aware attention identifies influential neighbors and relations. Second, SHAP-style attribution is computed on the final component feature representation or on a differentiable surrogate to identify the metrics that contributed most strongly. Third, explicit graph traversal converts high-influence entities into human-readable dependency paths. For example, the system can report that Pod analytics-api-17 shows rising memory usage and restart frequency, runs on Node-3, is selected by Service-A, and is upstream of three services whose latency is increasing.

A propagation score for edge u to v combines the source anomaly probability, learned attention, relation criticality, and temporal alignment. High-scoring paths are returned as candidate cascading-failure explanations. These paths are hypotheses for operator investigation rather than proof of causality.

$$R_{(u \rightarrow v)}^t = a_{\hat{u}}^t * \alpha_{uv}^t * C_r * D_{uv}^t \quad (14)$$

9. Training Objective and Online Inference

The training objective is multi-task and can combine anomaly classification, multi-horizon failure classification, discrete-time survival likelihood, reconstruction regularization, and temporal consistency. Focal loss or class-weighted binary cross-entropy is recommended because true failures are rare. Calibration must be evaluated separately because high discrimination does not guarantee reliable probabilities.

Online inference proceeds by updating the current graph, extracting the recent snapshot sequence, encoding typed nodes and relations, computing temporal states, generating risk estimates, and writing predictions back to the graph store. Alerts should use hysteresis and persistence rules to prevent oscillation. Model retraining should be triggered by scheduled intervals, drift indicators, or significant changes in platform architecture.

$$L_{\text{total}} = \beta_a L_{\text{anomaly}} + \beta_f L_{\text{failure}} + \beta_h L_{\text{hazard}} + \beta_c L_{\text{calibration}} + \beta_s L_{\text{smooth}} \quad (15)$$

10. Experimental Methodology

10.1 Testbeds

A rigorous evaluation should use at least two environments. The Kubernetes testbed should contain a multi-node cluster with several microservices, autoscaling, persistent storage, and observable service dependencies. The Hadoop testbed should contain NameNode and DataNode services, YARN scheduling, multiple workloads, and controllable storage and network faults. Prometheus-compatible monitoring should be synchronized with topology snapshots and incident timestamps.

To assess generalization, experiments should include both controlled laboratory workloads and at least one long-duration trace containing natural workload drift. The key objective is not merely to detect injected anomalies but to predict the transition from degradation to an operationally defined failure.

10.2 Fault scenarios

Kubernetes scenarios should include CPU saturation, memory leaks, OOMKilled, repeated restarts, failed health probes, Node memory pressure, disk pressure, network latency, packet loss, bandwidth limitation, broken service connectivity, replica loss, and controlled node drain. Hadoop scenarios should include DataNode heartbeat delay, disk saturation, HDFS under-replication, corrupt block events, NameNode RPC latency, NodeManager failure, queue overload, task retry storms, and JVM garbage-collection pressure.

Fault intensity should be varied to produce both abrupt and gradual degradation. Each experiment should record the fault start time, first observable symptom, first model alert, and final failure time. This enables measurement of early-warning lead time.

Table 3. Recommended evaluation metrics

Metric family	Purpose
AUROC and AUPRC	Discrimination under class imbalance
Precision, recall, F1	Operational alert quality
Brier score and ECE	Probability calibration
Lead time	How early the model warns before failure
C-index / time-dependent AUC	Survival and time-to-event quality
Top-1 / Top-3 root-cause accuracy	Diagnostic usefulness
Inference latency and memory	Online deployability

10.3 Data splitting and leakage prevention

Random row-wise splitting is inappropriate because adjacent time windows from the same incident are highly correlated. The dataset should be divided chronologically, and all windows associated with a specific injected or natural incident should remain in one partition. A 60-20-20 chronological split is a suitable starting point. Additional experiments should include leave-one-failure-type-out testing and cross-platform transfer to evaluate robustness to unseen events.

11. Baselines and Ablation Studies

Baselines should include flat Isolation Forest, One-Class SVM, XGBoost or Random Forest, LSTM autoencoder, standard GCN with static adjacency, GAL-MAD-style GAT-BiLSTM with fixed topology, and dynamic KG features with a non-deep detector. The principal ablations remove one architectural element at a time: dynamic topology, relation typing, the temporal encoder, network context, survival loss, and explainability regularization.

A particularly important direct comparison is static versus dynamic graph construction. Another is explicit knowledge-graph relations versus an artificial similarity graph. The proposed model should also be compared with a version that uses current anomaly prediction only, because this reveals whether the failure head adds real early-warning value rather than repackaging anomaly scores.

Table 4. Proposed ablation plan

Experiment	Change from full model	Question answered
A1	Static graph only	Are topology changes necessary?
A2	Homogeneous GAT	Do typed relations add value?
A3	No temporal encoder	Does degradation history improve forecasting?
A4	No KG, flat vectors	How much context is contributed by graph structure?
A5	No survival objective	Does time-to-event learning improve calibrated risk?
A6	No network edges	How important are runtime dependencies?

12. Feasibility Evidence from the Two Source Studies

The numerical observations in this section are prior evidence from the two source studies and are not results of DKG-TGAT-Forecast. Paper 3 showed that context-aware dynamic KG features can outperform non-contextual baselines in a benchmark containing difficult topology-driven faults. Its best reported online model reached an F1 score of 66.7% with precision of 78.6% and recall of 57.9%, while an ensemble increased recall to 89.5% at a cost in precision. These results demonstrate both the value of dynamic context and the challenge of balancing false alarms against missed faults.

Paper 4 reported that GAL-MAD achieved recall of 0.9884, 0.9228, and 0.8938 for 95:5, 90:10, and 60:40 normal-to-anomaly ratios, respectively. The ablation study showed lower recall after removing either the GAT or LSTM components, supporting the hypothesis that spatial and temporal modeling are complementary.

These findings motivate the proposed synthesis: Paper 3 supplies dynamic semantic context and Paper 4 supplies deep spatiotemporal learning. The proposed research question is whether their combination, plus a survival-style failure objective, can deliver calibrated early warnings in dynamic Kubernetes and Hadoop platforms.

13. Discussion

The proposed method changes the unit of analysis from an isolated metric sample to an evolving operational neighborhood. This is especially important in Kubernetes, where a Pod may disappear and be recreated under a new identity, and in Hadoop, where storage and compute failures can alter placement and replication relationships. Dynamic graph modeling preserves these context changes instead of treating them as unexplained distribution drift.

A major design trade-off is explainability versus representation capacity. Deep latent embeddings can discover complex interactions, but operators often need explicit explanations. The knowledge graph partially solves this by anchoring predictions to typed entities and relations. Attention and SHAP should therefore be presented alongside graph paths, not as standalone numerical scores.

Another trade-off is global accuracy versus timely warning. A model can achieve high classification accuracy by alerting close to failure, but such predictions may provide little operational value. Early-warning lead time and calibration should therefore be first-class metrics. A system that predicts a 70% one-hour failure risk should fail at approximately that frequency across comparable predictions.

Cross-platform transfer is promising but nontrivial. Kubernetes and Hadoop share abstract concepts such as compute, workload, storage, control, and dependency, yet their operational semantics differ. The safest initial strategy is to share upper-level encoders and training objectives while retaining platform-specific node types, relation types, and output calibration.

14. Limitations and Threats to Validity

The proposed unified model has not yet been implemented and experimentally validated, so no claims are made about its achieved performance. Controlled fault injection may not fully represent production incidents, and benchmark workloads may be simpler than enterprise deployments. Pseudo-causal propagation paths derived from attention and attribution remain hypotheses, not proof of causation. Large dynamic graphs may also impose memory and inference costs that require neighborhood sampling, graph partitioning, or hierarchical modeling.

Failure labels are another source of validity risk. A transient SLO violation, process restart, node loss, and storage corruption are different events. The evaluation must define failure types and prediction horizons precisely. Finally, natural incident data may contain incomplete labels and post-incident remediation actions that confound the model. These limitations should be explicitly reported in any empirical study.

15. Conclusion

This paper proposed DKG-TGAT-Forecast, a new framework for multi-horizon failure-rate prediction in Kubernetes and Hadoop platforms. The method synthesizes the dynamic, context-aware knowledge-graph representation of Paper 3 with the GAT-BiLSTM deep anomaly model and SHAP localization strategy of Paper 4. The new architecture represents the platform as a sequence of typed graph snapshots, applies relation-aware graph attention, models degradation with a temporal encoder, and produces calibrated failure probabilities, discrete hazards, and interval failure-rate estimates.

The key novelty is the shift from static anomaly detection to dynamic, context-aware failure forecasting. By combining semantic topology, deep spatiotemporal learning, explicit time-to-event objectives, and graph-grounded explanations, the framework provides a coherent research direction for proactive AIOps in complex big-data platforms. The next step is a reproducible implementation and controlled evaluation across Kubernetes and Hadoop testbeds with strict temporal splitting, calibration analysis, and root-cause validation.

References

- [1] P. Moens, B. Steenwinckel, F. Ongenaes, B. Volckaert, and S. Van Hoecke, "Toward Context-Aware Anomaly Detection for AIOps in Microservices Using Dynamic Knowledge Graphs," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 1970-1988, 2026, doi: 10.1109/TNSM.2026.3652304.
- [2] L. Akmeemana et al., "GAL-MAD: Towards Explainable Anomaly Detection in Microservice Applications Using Graph Attention Networks," *arXiv:2504.00058*, 2025.
- [3] P. Velickovic et al., "Graph Attention Networks," *International Conference on Learning Representations*, 2018.
- [4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [5] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *International Conference on Learning Representations*, 2017.
- [6] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [7] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating Explanations for Graph Neural Networks," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," *IEEE International Conference on Data Mining*, pp. 413-422, 2008.
- [9] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-World Challenges and Research Innovations," *IEEE/ACM International Conference on Software Engineering Companion*, pp. 4-5, 2019.
- [10] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," *ACM CCS*, pp. 1285-1298, 2017.
- [11] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *International Conference on Learning Representations*, 2015.
- [12] M. A. Rodriguez and P. Neubauer, "The Graph Traversal Pattern," *arXiv:1004.1001*, 2010.

- [13] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [14] J. Kosinska and M. Tobiasz, "Detection of Cluster Anomalies with Machine Learning Techniques," *IEEE Access*, vol. 10, pp. 110742-110753, 2022.
- [15] C.-W. Tien et al., "Kubanomaly: Anomaly Detection for the Docker Orchestration Platform with Neural Network Approaches," *Engineering Reports*, vol. 1, no. 5, e12080, 2019.